Yenepoya Institute of Technology
Moodbidri, Karnataka
Affiliated to Visvesvaraya Technological University, Belagavi

PROJECT REPORT

On

AI-Powered Exam Paper Solver

Intelligent Question-Answer Generation using Claude API

Submitted By
Muhammad Sabith Ismail
USN: 4DM25AI035
Department of Artificial Intelligence & Machine Learning
1st Year — Semester II

Academic Year: 2025–26

Abstract

The AI-Powered Exam Paper Solver is a full-stack web application designed to help students generate complete, module-wise solved exam papers on-demand. Traditional study methods require students to manually search through textbooks and notes to prepare answers for each module of their university syllabus. This project eliminates that bottleneck by integrating the Anthropic Claude Sonnet API directly into a React-based frontend to auto-generate structured Q-A content aligned with VTU course modules.

The application features a 3D tilt-card input interface where students enter their branch, course name, course code, and exam type. Upon submission, a five-module tab view is rendered; clicking any tab triggers a Claude API call that returns a formatted answer set including math blocks, section titles, and result stamps in the project's signature gold-and-navy academic theme. Generated modules are cached in React state so repeated tab clicks do not re-trigger API calls.

The backend is built with FastAPI (Python) to support future features such as user accounts and paper persistence, while the frontend is deployable to Vercel or Netlify via GitHub CI/CD. This project demonstrates a practical application of large language models (LLMs) in educational technology and establishes a scalable architecture that can be extended to multiple universities and course types.

Keywords: AI, Claude API, LLM, Exam Paper Generation, VTU, React, FastAPI, Anthropic, Educational Technology

Chapter 1 — Introduction

1.1 Background

University examinations, particularly those under the Visvesvaraya Technological University (VTU) framework, are structured around a five-module syllabus. Students are expected to produce detailed written answers covering all five modules across each subject in their semester. Preparing comprehensive answers for every module of every subject is a highly time-intensive process that often leads to uneven coverage, with students focusing only on modules they have already encountered in previous papers.

The rapid advancement of large language models (LLMs) has opened new possibilities in educational technology. Models such as Anthropic's Claude can generate contextually accurate, well-structured academic content when given appropriate prompts. This project leverages that capability to provide students with instant, syllabus-aligned solved papers for any course they are studying.

1.2 Motivation

The key motivators for this project include:

- Students spend disproportionate time searching for answers rather than understanding them.
- No existing tool generates module-wise solved papers aligned to VTU's specific course structure.
- LLMs are powerful enough to produce accurate academic answers with correct formatting.
- A well-designed UI can make AI-generated content accessible and credible for students.

1.3 Objectives

The primary objectives of this project are:

- To build a web application that generates fully solved VTU exam papers on demand.
- To integrate the Anthropic Claude Sonnet API for live Q-A generation per module.
- To design a clean, academic-style UI with accordion Q-cards and math block support.
- To implement a caching mechanism to avoid redundant API calls for already-generated modules.
- To provide a FastAPI backend scaffold for future features including user accounts and paper history.

Chapter 2 — System Design & Architecture

2.1 System Overview

The application is composed of two primary layers: a React + Vite frontend and a FastAPI Python backend. The frontend handles all UI interactions and makes direct calls to the Anthropic Claude API for content generation. The backend provides a REST API scaffold that currently handles configuration and is designed to be extended with authentication and data persistence in future iterations.

2.2 Architecture Diagram (Textual)

The high-level data flow is as follows:

- User fills the 3D input card (branch, course, code, exam type) and clicks Generate.
- The React frontend transitions to the paper view with five module tabs.
- On tab click, a structured prompt is constructed and sent to the Claude Sonnet API.
- Claude returns a JSON-structured response containing Q-A pairs with formatting markers.
- The frontend renders the response as accordion Q-cards with math blocks and section headers.
- The generated module is stored in React state; subsequent clicks on the same tab serve from cache.

2.3 Frontend Design

The frontend is built with React (Vite toolchain) and styled using Tailwind CSS. The design system uses two dominant colors — Navy (#0A1628) and Gold (#C9A84C) — to replicate the look of a formal academic answer booklet. The input page uses a CSS 3D perspective transform to create a tilt-card effect on mouse movement via Three.js event listeners, giving the UI a premium interactive feel.

The paper page is structured as follows:

- A fixed header showing the course name, code, and exam type.
- Five horizontally arranged module tabs, each showing a ✓ badge when its content has been generated.
- An accordion Q-card list for the active module — each card has a gold question header and an expandable answer block.
- Math blocks inside answers are rendered in a dedicated monospace block with light background.

2.4 Backend Design

The backend is a FastAPI application running on Python 3.11+. It exposes RESTful endpoints for configuration retrieval and is designed to be stateless. In future versions, the backend will handle JWT-based authentication, PostgreSQL-backed paper storage, and server-side prompt caching. The API is documented automatically via FastAPI's built-in OpenAPI/Swagger integration.

2.5 Claude API Integration

The Anthropic Claude Sonnet model is used for all content generation. Each API call is constructed with a carefully engineered system prompt that instructs Claude to:

- Respond only with Q-A content relevant to the specified module of the given course.
- Format each question as a bold header followed by a detailed answer block.
- Include mathematical derivations in code-fenced blocks when relevant.
- End each answer with a result statement for exam readability.
- Limit responses to a maximum of five questions per module for conciseness.

Chapter 3 — Technology Stack

3.1 Frontend Technologies

Technology	Version	Purpose
React	18.x	Component-based frontend UI framework
Vite	5.x	Fast build tool and development server
Tailwind CSS	3.x	Utility-first CSS for the gold/navy design system
Three.js	0.160+	3D tilt-card perspective effect on the input form

3.2 AI & API Technologies

- Anthropic Claude Sonnet (claude-sonnet-4-6) — Core LLM for Q-A generation
- Anthropic JavaScript SDK — Handles API authentication, request formatting, and streaming
- Prompt Engineering — Structured system prompts enforce module-aligned academic output

3.3 Backend Technologies

- FastAPI 0.110+ — Python REST framework with automatic OpenAPI documentation
- Python 3.11+ — Runtime for the backend server
- Uvicorn — ASGI server for FastAPI

3.4 DevOps & Deployment

- GitHub — Version control and source hosting
- Vercel / Netlify — Static frontend deployment with automatic CI/CD from main branch
- GitHub Actions (planned) — Automated test and lint pipelines

Chapter 4 — Implementation

4.1 Input Interface

The input interface is implemented as a single React component (InputCard) that uses CSS custom properties and JavaScript mousemove event listeners to compute a 3D perspective transform. As the user moves their mouse over the card, the X and Y rotations are calculated relative to the card's bounding rectangle and applied as a CSS transform with a cubic-bezier transition for smoothness.

The form collects four fields:

- Branch — dropdown with all VTU engineering branches
- Course Name — free text input (e.g., 'Quantum Physics and Applications')
- Course Code — alphanumeric code (e.g., '1BPHYS102')
- Exam Type — dropdown: Internal Assessment, End-Semester, or Practice

4.2 Prompt Construction

When a module tab is clicked, the frontend constructs a structured prompt using the input metadata and the module number. The system prompt sets the role as an expert academic answer writer for VTU engineering students. The user message specifies the course, code, module number, and exam type. The model is instructed to return five questions with detailed answers, including derivations in math blocks.

4.3 Response Parsing & Rendering

The Claude API response is a markdown-style text block. The frontend parser identifies question headers (lines starting with 'Q' or bold markers), answer blocks, and code/math fences. Each parsed Q-A pair is stored as an object in React state and rendered as an accordion card. Math blocks are rendered in a fixed-width, background-highlighted container to distinguish them from prose text.

4.4 Caching Strategy

Generated modules are stored in a React useRef map keyed by module number. Before any API call, the frontend checks this map; if a cached result exists, it is rendered immediately without an API round-trip. The cache is session-scoped (in-memory) and is cleared when the user clicks 'New Paper', resetting the application to the input state.

4.5 FastAPI Backend

The backend exposes a /config endpoint returning the application configuration (model version, max tokens, prompt templates). It is structured as a standard FastAPI app with CORS middleware configured to allow requests from the Vite dev server (localhost:5173) and the production domain. Future endpoints planned include /papers (CRUD for saved papers) and /auth (JWT login and registration).

Chapter 5 — Results & Discussion

5.1 Functional Outcomes

The application successfully achieves all stated objectives:

- Students can enter any VTU course name and code and receive a fully solved five-module paper within seconds.
- Each module generates five well-structured Q-A pairs with correct academic formatting.
- Math-heavy subjects (Physics, Maths, Circuit Theory) produce equations in dedicated code blocks.
- Module tabs display ✓ badges on completion and do not re-fetch on repeated clicks.
- The 3D tilt-card input and gold/navy paper design deliver a professional, polished user experience.

5.2 Performance

API response latency for a single module call averages 3–6 seconds depending on answer length and Claude's current load. The caching mechanism ensures that subsequent accesses to the same module are instantaneous. The React frontend renders the parsed Q-A list within 50 ms of receiving the API response.

5.3 Limitations

- The application requires an active internet connection and a valid Anthropic API key.
- API costs scale with usage; high-volume deployment would require a token budget strategy.
- Claude may occasionally generate generic answers for highly specialized or niche courses.
- Math rendering is plaintext-based; a dedicated LaTeX renderer (e.g., KaTeX) would improve readability.

5.4 Testing

The application was tested on the following course types:

- Quantum Physics and Applications (1BPHYS102) — Physics, math-heavy
- Engineering Mathematics II (1BMATC201) — Pure mathematics, derivations
- Python Programming (1BECM206) — Computer science, algorithmic content
- Engineering Chemistry (1BCHEM102) — Chemistry, structural diagrams (text-described)

All four courses produced coherent, module-aligned answers. Physics and Mathematics modules correctly included formula derivations in math blocks. Programming modules included pseudocode and complexity analysis.

Chapter 6 — Future Scope

The following enhancements are planned for future versions of the application:

6.1 PDF Export

A one-click PDF export feature will allow students to save the generated solved paper as a professionally formatted PDF document, ready for printing or offline study.

6.2 User Accounts & Paper History

Implementing JWT-based authentication and a PostgreSQL backend will allow students to create accounts, save generated papers, and build a personal library of solved exam content organized by subject and semester.

6.3 Multi-University Support

The prompt templates can be parameterized to support syllabi from other Indian universities such as SPPU (Pune), Mumbai University, and Anna University — making the tool useful for a much wider student base.

6.4 Previous Year Paper Mode

Students will be able to upload actual previous-year question papers as PDFs. The system will parse the questions and use Claude to generate step-by-step answers for each one, creating a fully solved reference paper.

6.5 LaTeX Math Rendering

Integrating the KaTeX JavaScript library will enable proper typeset rendering of mathematical equations, replacing the current plaintext math blocks with publication-quality formulae.

6.6 Offline Mode with Local LLM

For users without internet access or API budget constraints, a local LLM integration (e.g., Ollama with Llama 3.1) will be explored to provide offline generation capability on student laptops.

Chapter 7 — Conclusion

The AI-Powered Exam Paper Solver demonstrates that large language models can be effectively integrated into practical educational tools with a well-designed UI and structured prompt engineering. By combining the Anthropic Claude Sonnet API, a React frontend, and a FastAPI backend, this project delivers a functional, deployable product that genuinely reduces the time students spend on exam preparation.

The project is technically sound — leveraging modern JavaScript tooling (Vite, React 18), a Python REST backend (FastAPI), and responsible API usage (caching, structured prompts). The design system mirrors the formality of an academic answer booklet while maintaining a modern, interactive feel through 3D input cards and tabbed module navigation.

Most importantly, it establishes a scalable architecture. Every feature planned for future versions — PDF export, user accounts, multi-university support, LaTeX rendering — can be added without redesigning the core application. This makes the AI-Powered Exam Paper Solver a strong foundation for a broader educational technology platform that can serve thousands of VTU students in the years ahead.

Muhammad Sabith Ismail | 4DM25AI035 | Yenepoya Institute of Technology, Moodbidri

References

- Anthropic. (2024). Claude API Documentation. https://docs.anthropic.com
- React Team. (2024). React 18 Release Notes. https://react.dev/blog
- Holovaty, A., & Willison, S. (2023). FastAPI — Modern Web APIs with Python.
- Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. arXiv:2005.14165
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017.
- VTU. (2024). Scheme and Syllabus — B.E./B.Tech. 2024-25. Visvesvaraya Technological University, Belagavi.
- Three.js Authors. (2024). Three.js Fundamentals. https://threejs.org/docs
- Tailwind CSS Team. (2024). Tailwind CSS v3 Documentation. https://tailwindcss.com/docs